

The multithreaded RTA-MM-GPR node

What changed on the multithreaded branch, how to run it, and what it measured

2026-09-27

Table of contents

1	Summary	1
2	How to run	2
2.1	Environment notes (PX4 v1.17 / Jazzy / this machine)	3
3	Architecture	3
3.1	Rollout backends	4
3.2	Time-indexed reference	4
4	Bugs found and fixed	5
4.1	GP never received wind data	5
4.2	LQR re-linearised every tick	5
4.3	Crash when offboard engaged before the first odometry message	5
4.4	Stalls that aren't the rollout	5
5	Results	6
5.1	Plan freshness and the tube horizon	8
6	The new flight recorder	8
7	Suggested next steps	10

1 Summary

The node's control loop used to share one thread with the reachability rollouts, a stop-the-world garbage collector, in-flight JIT compilation, and hundreds of `prints` per second. On this machine that meant **6–12 Hz instead of 100 Hz** during the RTA phase, and every original flight in the benchmark crashed. On the `multithreaded` branch:

- the control law, rollouts, estimation, odometry and PX4 I/O run in **separate callback groups** on a **MultiThreadedExecutor** (or, optionally, the rollout runs in its **own process**);
- the controller reads **immutable snapshots** (state, plan, gains) swapped in atomically;

- GC pauses, in-flight JIT compiles and hot-path printing are gone;
- a **new flight recorder** stores every rollout plan once, in full precision, in one HDF5 file (plus a CSV in the old ROS2Logger layout).

Measured over three SITL flights per configuration, control ran at **99.8 Hz with a p99 period of 13–14 ms** (events + process), against 6 Hz before. Section 5 also reports what is *not* fixed: a residual failure rate in the aggressive descent manoeuvre that is a control-design question, not a scheduling one.

Background on executors, callback groups, the GIL, processes and GC pauses is in the companion document, *Concurrency in ROS 2 (Python)*.

2 How to run

```
# terminal 1: PX4 SITL (Gazebo Harmonic, x500)
cd ~/PX4-Autopilot && make px4_sitl gz_x500
# terminal 2: XRCE-DDS agent
MicroXRCEAgent udp4 -p 8888
# terminal 3: relay (remaps PX4 1.17's versioned topic) + node
source ~/ws_sensor_combined/install/setup.bash && source install/setup.bash
ros2 launch px4_rta_mm_gpr sim_relay_rta_launch.py          # defaults
# or directly:
ros2 run px4_rta_mm_gpr px4_rta_mm_gpr --sim --log-file run1.log \
    --executor events --rollout-backend process --tube-horizon 3
```

flag	default	meaning
--executor {multi,single,events}	multi	MultiThreadedExecutor (one thread per callback group), the original SingleThreadedExecutor, or rclpy's C++ EventsExecutor
--threads N	#groups (5)	worker threads for multi
--rollout-backend {thread,process}	thread	rollout in its callback group's thread, or in a spawned worker process
--rollout-cpus 2,3	none	CPU affinity for the worker process
--tube-horizon S	30	rollout horizon in seconds (compute time horizon; see Section 5.1)
--gp-learn / --no-gp-learn	on	feed wind estimates into the GP (bug fix, see Section 4.1)
--gc-freeze / --gc-no-full	on / on	garbage-collector settings (Section 4.4)

<code>--verbose</code>	off	the original per-callback printing (slow)
------------------------	-----	--

Recommended on this machine: `--executor events --rollout-backend process`. Since round 3, rollouts stop at the certification point, so `--tube-horizon` is only an upper bound. See document 03 for the round-3 flags (`--nr-ref-from-plan`, `--nr-anti-windup`, `--tube-margin`, `--replan-lead`, `--thrust-limits-mass-scaled`, `--entry-ramp`) and for the cause of the crashes. Document 04 covers the altitude offset (`--gp-feedforward`), the ground floor (`--min-altitude`) and the LAND backup (`--backup`). The defaults keep the paper’s 30 s horizon and the plain multithreaded executor, so nothing changes unless you ask for it.

2.1 Environment notes (PX4 v1.17 / Jazzy / this machine)

- PX4 1.17 publishes `/fmu/out/vehicle_local_position_v1`. `mocap_px4_relays/full_state_relay` still subscribes to the unversioned name, so its freshness gate never opens and no `FullState` is published. The launch files now remap the topic.
- SITL without `QGroundControl` refuses to arm (“No connection to the GCS”): `param set NAV_DLL_ACT 0`.
- `immrax 0.3.x` pins `numpy` to `2.3`, but ROS 2 Jazzy’s message bindings are built against Ubuntu’s `numpy 1.26`. `immrax` is installed with `--no-deps` and its other dependencies with `numpy` pinned at `1.26.4` (all in `~/.local`). `scipy` `>=1.14` is needed for `Rotation.from_quat(..., scalar_first=True)`.
- ROS2Logger renamed its package `Logger` → `ros2_logger`; both are accepted.

3 Architecture

callback group	callbacks	rate
<code>px4_io_group</code>	offboard heartbeat, <code>vehicle_status</code> , RC channels, 1 Hz status line	10 Hz
<code>state_group</code>	<code>FullState</code> odometry → <code>VehicleState</code> snapshot	40 Hz
<code>control_group</code>	NR tracker + RTA feedback → body-rate setpoint	100 Hz
<code>rollout_group</code>	install finished rollouts; start a new one when the plan expires	100 Hz check
<code>estimation_group</code>	wind EKF → GP data; LQR re-linearisation check	10 Hz / 20 Hz

Data flows between groups only through objects that are **replaced, never mutated**:

shared object	writer	readers
<code>self.state: VehicleState</code> (frozen dataclass)	odometry	control, wind, LQR, rollout
<code>self.plan: RolloutPlan</code> (frozen dataclass of NumPy arrays)	rollout	control
<code>self.gains = (K_feedback, K_reference)</code>	LQR	control, rollout
<code>self.gz_wind_obs_in_y / gy_wind_obs_in_z</code> (copy-on-write arrays)	wind	rollout
<code>self.last_input</code>	control	wind, LQR

3.1 Rollout backends

`px4_rta_mm_gpr/concurrency/rollout_backends.py` puts the rollout behind a two-call interface:

```
backend.submit(RolloutRequest(t_start, state, K_feedback, K_reference,
    ↪ obs_wy, obs_wz, ...))
result = backend.poll()    # RolloutResult or None
```

`ThreadRolloutBackend` computes inside `submit` (blocking only the rollout group’s thread). `ProcessRolloutBackend` sends the request to a `spawned` worker that owns its own JAX runtime and returns results through a pipe. `submit` and `poll` never block, so the process backend keeps the controller at 100 Hz even with a single-threaded executor. Both backends produced the same plan for the same request (`np.allclose` over the whole tube and reference).

3.2 Time-indexed reference

The controller used to step through a plan with a tick counter (`traj_idx += 1`, reset to 0 when a plan was installed). That’s only correct if the controller runs exactly at 100 Hz *and* the plan is installed at the instant it was computed for. Neither holds once rollouts run concurrently: a plan computed from the state at time t_0 is installed about 25–200 ms later. The controller now asks the plan for the row that corresponds to *now*:

```
traj_idx = plan.index_at(t)    # int((t - plan.t_start) / plan.dt), clamped
```

4 Bugs found and fixed

4.1 GP never received wind data

```
self.gz_wind_obs_in_y.at[wind_idx, :].set(jnp.array(...))    # original
```

JAX arrays are immutable: `.at[...].set(...)` *returns* a new array, and here that array was thrown away. The GP data buffers therefore kept their initial constant values for the whole flight, and the rollouts never used a single wind estimate. The buffers are now NumPy arrays updated copy-on-write. `--no-gp-learn` restores the old behaviour (for example, to reproduce the paper's runs).

4.2 LQR re-linearised every tick

The trigger was `abs(self.yaw) > max_yaw_stray (5°)`: the *absolute* yaw, and `adjust_yaw` unwraps yaw into an accumulated angle. With a non-zero desired yaw, or after any full turn, the condition is permanently true, and the node re-linearised and re-solved two Riccati equations (~ 4 ms) on *every* control tick. The flight log of one run showed 349 LQR updates in 19 s of RTA flight. The trigger now uses the tracking error to the desired yaw from `get_ref()`, wrapped to $[-\pi, \pi]$, and the update runs in the estimation group at most at 20 Hz, so it never delays a control tick.

4.3 Crash when offboard engaged before the first odometry message

The wind callback read `self.nr_state_vector` before the odometry callback had ever set it (`AttributeError` 1.1 s after start). Callbacks now return early until the first `VehicleState` exists, and the controller holds the pre-RTA setpoint until the first plan and gains exist.

4.4 Stalls that aren't the rollout

stall	size	fix
full garbage collections (logger lists)	$4\text{--}5 \times \sim 300$ ms per flight	<code>gc.freeze()</code> after init, no automatic gen-2 in flight, recorder without per-tick lists
JIT recompiles on the first RTA ticks	$322 + 43 + 60$ ms	warm up with the controller's own output fed back (<code>JAX_LOG_COMPILE=1</code> to check)
printing + JAX indexing of plans	~ 9 ms of every 10 ms tick	<code>--verbose</code> only; plans stored as NumPy

5 Results

Setup: PX4 v1.17.0 SITL, Gazebo Harmonic x500, this machine (14 cores). Each configuration was flown three times from a fresh SITL instance. Rows below are the RTA phase ($15 \leq t < 35$ s); the statistics come from the logged control-tick timestamps. “Crashed” means $z > 0$ (NED) at some point, i.e. ground contact. The first four configurations ran with `JAX_LOG_COMPILE=1` (to confirm that nothing compiles in flight any more), which adds a little logging overhead.

configuration	control rate	p99 period	ticks > 15 ms	rollout	crashed
original (single-threaded)	5.8–6.4 Hz	248–394 ms	89–100 %	186–190 ms	3/3
refactor, single executor	24.5–27.2 Hz	221–224 ms	17–19 %	178–187 ms	3/3
refactor, multi + thread	85.6–90.5 Hz	36–54 ms	6–10 %	165–201 ms	1/3
refactor, events + process	99.5–99.9 Hz	13–14 ms	< 1 %	167–193 ms	2/3
multi + thread, 3 s horizon	79.9–99.4 Hz	17–49 ms	2–16 %	14–30 ms	1/3
events + process, 3 s horizon	99.8 Hz	12–17 ms	0.2–3 %	25–32 ms	1/3

The full per-run table is in `docs/data/summary_table.txt`, and the per-tick data in `docs/data/`.

Reading the results honestly:

- **The scheduling problem is solved.** The control loop gets its 100 Hz whenever the rollout is off its thread. The process backend is the most robust, because the rollout’s Python glue, its memory allocation and its JIT runtime can’t touch the controller’s interpreter.
- **Separating the threads is necessary, not sufficient.** In every configuration, the vehicle enters the RTA phase at $(y, z) = (3.9, -12.5)$ with the reference pulling it toward `GOAL_STATE = (0, -0.6)`. The roll rate saturates at ± 1 rad/s, and the vehicle descends at about 2 m/s while y oscillates. When y is still swinging as the ground arrives, the run crashes, and once the attitude is extreme, `adjust_yaw` miscounts wraps and the NR tracker commands yaw rates above 100 rad/s. This happens with near-perfect timing, so it’s a property of the manoeuvre and gains, not of the executor. Three runs per configuration are too few to rank the fast configurations against each other on crash rate.

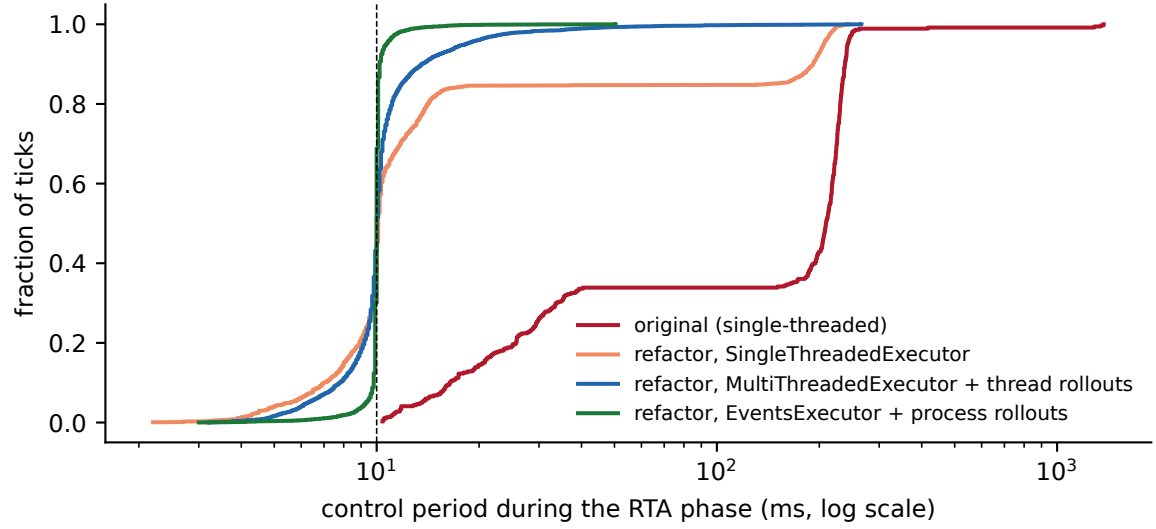


Figure 1: Distribution of the control period during the RTA phase, all runs pooled. The dashed line is the 10 ms target. The original node spends almost the whole RTA phase at ~ 200 ms periods; the refactored single-threaded node is fixed except while a rollout runs; the multithreaded and events + process configurations sit on the target.

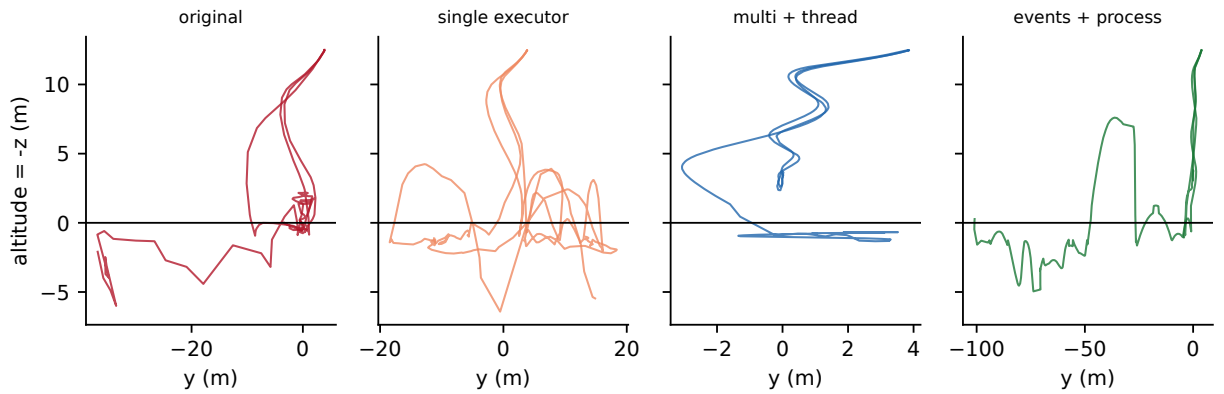


Figure 2: Flown paths in the y -altitude plane (three runs each, RTA phase). The black line is the ground.

5.1 Plan freshness and the tube horizon

With a 30 s horizon, a rollout (3000 scan steps) takes ~ 190 ms, while the safety horizon it certifies is only 0.1–0.3 s. So on 58–74 % of control ticks the controller follows a plan past its certified horizon (`plan_expired`). The interval over-approximation diverges within about a second (Figure 4 (b)), so nothing past the safety horizon is ever used.

The rollout is a causal `lax.scan`, so its first N steps don't depend on the horizon length: **a 3 s horizon produces exactly the same plan whenever the violation lies inside it**, and computes it about $6\text{--}7\times$ faster (25–32 ms). The expired fraction drops to 7–14 % (Figure 3). If no violation is found within the horizon (`violation_idx = -1`), the code replans immediately, exactly as before.

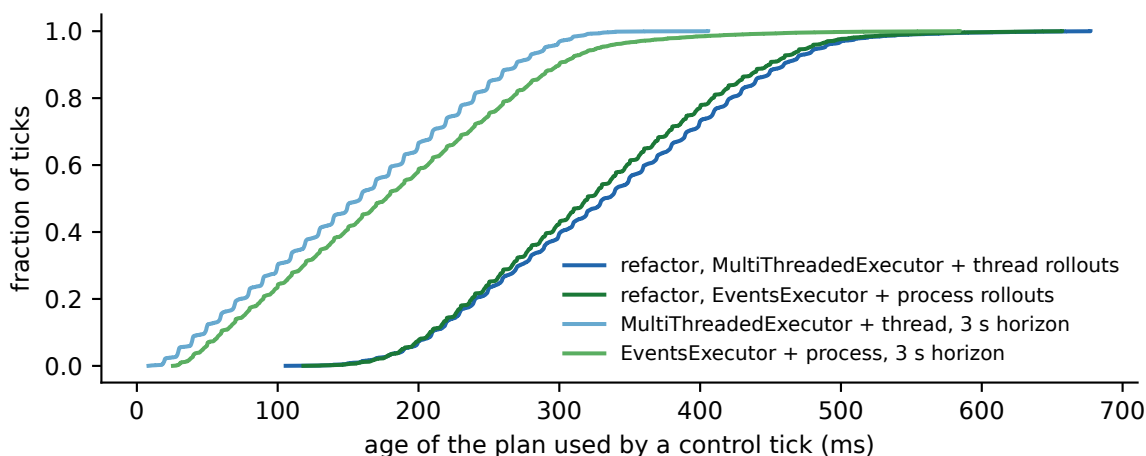


Figure 3: Age of the plan used by each control tick (time since its rollout started).

6 The new flight recorder

`px4_rta_mm_gpr/flight_log/` replaces the ROS2Logger LogTypes:

- **In flight:** one preallocated float64 row per control tick (`ColumnBuffer`, grows by doubling), written in place, **5.7 μ s per tick**, with no Python containers kept. The GC-tracked heap at the end of a flight went from **24 080 objects (ROS2Logger)** to **470–1 415**.
- **Plans:** stored **once, when installed**, by reference (they are immutable), instead of re-appending a 12×4 slice of the tube on every tick. Each plan keeps its complete tube, reference and feedforward in float64, **plus its inputs** (initial state, both gain matrices, both GP data buffers), so any plan can be recomputed offline with `RolloutEngine`.
- **At shutdown:** one HDF5 file (gzip + shuffle; about 3 MB for a flight with 82 plans) and a CSV in the exact ROS2Logger column layout, reconstructed from the stored plans, so the existing notebooks keep working. ROS2Logger is still used to choose the output directory and to copy its notebooks.

```

flight.h5
  attrs      platform, executor, backend, gp_learn, tube_horizon,
            Q/R weights, input limits, git commit, host, date, ...
  ticks/     time, state (x..pitch), command (thrust..yaw_rate),
            ctrl_comp_time, control_period, wy, wz,
            plan_seq, traj_idx, plan_age, plan_expired,
            rollout_latency, rollout_comptime,
            ref_{py,pz,h,v,theta}, tube_{py,pz}_{lo,hi}
  wind/      time, wy, wz, ay/az measured and modelled, heights
  gains/     time, first_lqr, K_feedback_ij, K_reference_ij
  plans/00001/ reachable_tube, rollout_ref, feedfwd_input,
            state0, K_feedback, K_reference, obs_wy, obs_wz
            attrs: t_start, dt, collection_time, violation_idx,
                  compute_time, latency, warmup
  timing/    loop periods and exec times, rollout compute/latency,
            GC pauses per generation

```

```

from px4_rta_mm_gpr.flight_log import FlightLog
with FlightLog('run1.h5') as log:
    log.ticks                                # pandas DataFrame
    log.plan(27)['reachable_tube'] # (N+1, 10) lower/upper embedding states,
    ↪ float64
    log.tube_from(21.3)                    # the tube the controller was relying on
    ↪ at t = 21.3 s
    log.to_legacy_csv('run1.csv')

```

From the shell: `ros2 run px4_rta_mm_gpr flight_log run1.h5 [--csv run1.csv]`.

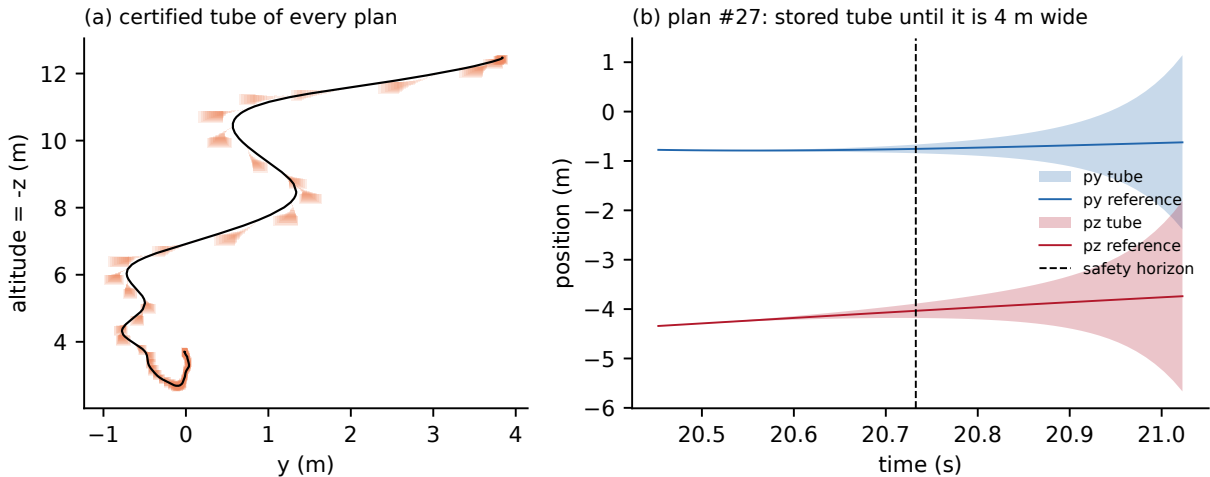


Figure 4: From a stored flight log: (a) the certified part of every plan's tube along the flown path; (b) one plan's full stored tube versus time, cut where it becomes 4 m wide. The interval bound grows quickly after the dashed safety horizon.

7 Suggested next steps

1. **Use `--tube-horizon 3`** (or less) for flights: the same plans, about $6\text{--}7\times$ less compute.
2. **Saturate the NR yaw-rate command** and make `adjust_yaw` robust (use the quaternion's yaw directly, or wrap the *error* rather than unwrapping the angle). Commands above 100 rad/s were logged after upsets.
3. **Soften the RTA entry transient.** Start the RTA phase from a reference near the current state, or ramp `GOAL_STATE`, instead of switching from $(3.9, -12.5)$ straight to $(0, -0.6)$ with saturated roll rates.
4. Run each configuration more times (10+) before comparing crash rates between the fast configurations.